EXPERIMENT 7:

Mansha Bhatnagar
1024150192
2O23

Topics:
A. Polymorphism
B. Virtual Function and Pure Virtual Function
C. Operator Overloading

1. Class polygon contains data member width and height and public method set_value() to assign values to width and height. Class Rectangle and Triangle are inherited from polygon class. Both the classes contain public method calculate_area() to calculate the area of Rectangle and Triangle. Use base class pointer to access the derived class object and show the area calculated.

ass 7 Q1.cpp | ass7Q1.cpp

D: > COLLEGE > oops > ass7 > ass7Q1.cpp > main()

```cpp
#include <iostream>
using namespace std;
class Polygon {
protected:
    float width, height;
public:
    void set_value(float w, float h) {
        width = w;
        height = h;
    }
    virtual float calculate_area() = 0;
};
class Rectangle : public Polygon {
public:
    float calculate_area() {
        return width * height;
    }
};
class Triangle : public Polygon {
public:
    float calculate_area() {
        return 0.5 * width * height;
    }
};
int main() {
    Rectangle rect;
    Triangle tri;
    Polygon *p1 = &rect;
    Polygon *p2 = &tri;
    p1->set_value(10, 5);
    p2->set_value(10, 5);
    cout << "Rectangle Area: " << p1->calculate_area() << endl;
    cout << "Triangle Area: " << p2->calculate_area() << endl;
    return 0;
}
```

CHAT TERMINAL Code

```
PS C:\Users\HP> cd "C:\Users\HP\AppData\Local\Temp\" ; if ($?) {
g++ tempCodeRunnerFile.cpp -o tempCodeRunnerFile } ; if ($?) { .\
tempCodeRunnerFile }
Rectangle Area: 50
Triangle Area: 25
PS C:\Users\HP\AppData\Local\Temp>
```

2. Write a program to create a class shape with functions area and display to find area and display the name of the shape and other essential component of the class. Create derived classes circle, rectangle and triangle each having overridden functions area and display. Write a program to find and display the area of circle, rectangle and triangle.

```cpp
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void area() = 0;
    virtual void display() = 0;
};

class Circle : public Shape {
    float r;
public:
    Circle(float radius) : r(radius) {}
    void area() { cout << "Area: " << 3.14 * r * r << endl; }
    void display() { cout << "Shape: Circle" << endl; }
};

class Rectangle : public Shape {
    float l, b;
public:
    Rectangle(float length, float breadth) : l(length), b(breadth) {}
    void area() { cout << "Area: " << l * b << endl; }
    void display() { cout << "Shape: Rectangle" << endl; }
};

int main() {
    Circle c(5);
    Rectangle r(4, 6);

    c.display();
    c.area();

    r.display();
    r.area();

    return 0;
}
```

```
PS C:\Users\HP> cd
g++ tempCodeRunner
tempCodeRunnerFile
Rectangle Area: 56
Triangle Area: 25
PS C:\Users\HP\App
Temp\" ; if ($?) {
le } ; if ($?) { .
Shape: Circle
Area: 78.5
Shape: Rectangle
Area: 24
PS C:\Users\HP\App
```

3. Write a C++ program to compute area of right angle triangle, equilateral triangle, isosceles triangle using function overloading concept.

D: > COLLEGE > oops > ass7 > ass7Q3.cpp > main()

```cpp
#include <iostream>
#include <cmath>
using namespace std;

void area(float base, float height) {
    cout << "Right Angle Triangle Area: " << 0.5 * base * height << endl;
}

void area(float side) {
    cout << "Equilateral Triangle Area: " << (sqrt(3) / 4) * side * side << endl;
}

void area(double side, double base) {
    double h = sqrt((side * side) - (base * base / 4));
    cout << "Isosceles Triangle Area: " << 0.5 * base * h << endl;
}

int main() {
    area(10.0f, 5.0f);
    area(6.0f);
    area(10.0, 8.0);
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\App
g++ tempCodeRunnerFile.cpp -o tempC
tempCodeRunnerFile }
Right Angle Triangle Area: 25
Equilateral Triangle Area: 15.5885
Isosceles Triangle Area: 36.6606
PS C:\Users\HP\AppData\Local\Temp>
```

4. Write a program with Student as abstract class and create derive classes Engineering, Medicine and Science from base class Student. Create the objects of the derived classes and process them and access them using array of pointer of type base class Student.

D: > COLLEGE > oops > ass7 > ass7Q4.cpp > main()

```cpp
#include <iostream>
using namespace std;

class Student {
public:
    virtual void process() = 0;
};

class Engineering : public Student {
public:
    void process() { cout << "Processing Engineering Student" << endl; }
};

class Medicine : public Student {
public:
    void process() { cout << "Processing Medicine Student" << endl; }
};

class Science : public Student {
public:
    void process() { cout << "Processing Science Student" << endl; }
};

int main() {
    Student* list[3];
    list[0] = new Engineering();
    list[1] = new Medicine();
    list[2] = new Science();

    for(int i = 0; i < 3; i++) {
        list[i]->process();
    }
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\Ap
nnerFile.cpp -o tempCodeRunnerFile
Processing Engineering Student
Processing Medicine Student
Processing Science Student
PS C:\Users\HP\AppData\Local\Temp>
```

5. Create a class Time with three private variables int h,m,s; Create a function to overload '+' operator to add two time variables.

```
int main(){
Time t1(5,15,34),t2(9,53,58),t3;
t3 = t1 + t2;
t3.show();
}
```

```cpp
#include <iostream>
using namespace std;

class Time {
    int h, m, s;
public:
    Time(int hr = 0, int min = 0, int sec = 0) : h(hr), m(min), s(sec) {}

    Time operator+(Time t) {
        Time temp;
        temp.s = s + t.s;
        temp.m = m + t.m + (temp.s / 60);
        temp.s %= 60;
        temp.h = h + t.h + (temp.m / 60);
        temp.m %= 60;
        return temp;
    }

    void show() {
        cout << h << ":" << m << ":" << s << endl;
    }
};

int main() {
    Time t1(5, 15, 34), t2(9, 53, 58), t3;
    t3 = t1 + t2;
    t3.show();
    return 0;
}
```

```
} ; if (
15:9:32
PS D:\COL
```

6. Define a class STRING and overload == to compare two strings and + operator for concatenation of two strings.

```cpp
#include <iostream>
#include <string.h>
using namespace std;

class STRING {
    char str[100];
public:
    STRING(const char* s = "") { strcpy(str, s); }

    bool operator==(STRING s) {
        return strcmp(str, s.str) == 0;
    }

    STRING operator+(STRING s) {
        STRING temp;
        strcpy(temp.str, str);
        strcat(temp.str, s.str);
        return temp;
    }

    void display() { cout << str << endl; }
};

int main() {
    STRING s1("Hello"), s2("World"), s3;
    s3 = s1 + s2;
    s3.display();
    if(s1 == s2) cout << "Equal"; else cout << "Not Equal";
    return 0;
}
```

```
PS C:\Users\H
 } ; if ($?)
15:9:32
PS D:\COLLEGE
empCodeRunner
}
HelloWorld
Not Equal
PS C:\Users\H
```

7. Write a program in C++ to create a class matrix and overload * operator using friend function to multiply two matrices.

ass7Q5.cpp | #include <iostream> Untitled-1 | #include <iostream> Untitled-2

```cpp
#include <iostream>
using namespace std;

class Matrix {
    int mat[2][2];
public:
    void input() {
        for(int i=0; i<2; i++)
            for(int j=0; j<2; j++) cin >> mat[i][j];
    }
    void display() {
        for(int i=0; i<2; i++) {
            for(int j=0; j<2; j++) cout << mat[i][j] << " ";
            cout << endl;
        }
    }
    friend Matrix operator*(Matrix m1, Matrix m2);
};
Matrix operator*(Matrix m1, Matrix m2) {
    Matrix res;
    for(int i=0; i<2; i++) {
        for(int j=0; j<2; j++) {
            res.mat[i][j] = 0;
            for(int k=0; k<2; k++)
                res.mat[i][j] += m1.mat[i][k] * m2.mat[k][j];
        }
    }
    return res;
}
int main() {
    Matrix a, b, c;
    cout << "Enter Matrix A (2x2): "; a.input();
    cout << "Enter Matrix B (2x2): "; b.input();
    c = a * b;
    c.display();
    return 0;
}
```

CHAT TERMINAL

```
PS C:\Users\HP> cd "C:\Users\HP\AppDa
nnerFile.cpp -o tempCodeRunnerFile }
Enter Matrix A (2x2): 1 2 3 4
Enter Matrix B (2x2): 7 8 9 10
25 28
57 64
PS C:\Users\HP\AppData\Local\Temp>
```

8. Overload '[]' to check array index out of bounds problem at run time.

#include <iostream> Untitled-1

```cpp
#include <iostream>
using namespace std;

class SafeArray {
    int arr[5];
public:
    SafeArray() {
        for(int i=0; i<5; i++) arr[i] = i * 10;
    }
    int& operator[](int index) {
        if(index < 0 || index >= 5) {
            cout << "Index out of bounds!" << endl;
            exit(0);
        }
        return arr[index];
    }
};

int main() {
    SafeArray a;
    cout << a[2] << endl;
    cout << a[10] << endl;
    return 0;
}
```

CHAT TERMINAL

```
PS C:\Users\HP> cd
nnerFile.cpp -o tem
20
Index out of bounds
PS C:\Users\HP\AppD
```

9. Overload '()' to input arbitrary number of input arguments for an object.

#include <iostream> Untitled-1 | #include <iostream> Untitled-2

```cpp
#include <iostream>
using namespace std;

class Adder {
public:
    int operator()(int a, int b) { return a + b; }
    int operator()(int a, int b, int c) { return a + b + c; }
};

int main() {
    Adder add;
    cout << add(10, 20) << endl;
    cout << add(10, 20, 30) << endl;
    return 0;
}
```

CHAT

```
PS C:
o tem
30
60
PS C:
```

10. Write a program in C++ to overload input operator (>>) and output operator (<<).

```cpp
#include <iostream>
using namespace std;

class Point {
    int x, y;
public:
    friend istream& operator>>(istream& in, Point& p) {
        in >> p.x >> p.y;
        return in;
    }
    friend ostream& operator<<(ostream& out, Point& p) {
        out << "(" << p.x << ", " << p.y << ")";
        return out;
    }
};

int main() {
    Point p;
    cin >> p;
    cout << p;
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppDa
pCodeRunnerFile } ; if ($?) { .\tempC
4
4566
(4, 4566)
PS C:\Users\HP\AppData\Local\Temp>
```

11. Write a program to convert basic data type (float) to user defined data type (object).

```
class Test {
private:
   //...
public:
   Test(data_type) {
      // conversion code
   }
};
```

#include <iostream> Untitled-3 | #include <iostream> Untitled-4

```cpp
#include <iostream>
using namespace std;

class Distance {
    int km, m;
public:
    Distance(float val) {
        km = (int)val;
        m = (val - km) * 1000;
    }
    void show() { cout << km << "km " << m << "m" << endl; }
};

int main() {
    float raw = 5.75;
    Distance d = raw;
    d.show();
    return 0;
}
```

CHAT TERMINAL

```
PS C:\Users\HP> cd "C:\Users\HP\A
 tempCodeRunnerFile } ; if ($?) {
5km 750m
PS C:\Users\HP\AppData\Local\Temp
```

12. Write a program to convert UDT to basic data type (float)

```
class Test {
public:
   operator data_type() {
      // conversion code
   }
};
```

```cpp
#include <iostream>
using namespace std;

class Weight {
    int kg, g;
public:
    Weight(int k, int gram) : kg(k), g(gram) {}
    operator float() {
        return kg + (g / 1000.0);
    }
};

int main() {
    Weight w(5, 500);
    float total = w;
    cout << total << " kg" << endl;
    return 0;
}
```

```
PS C:\Users\H
 tempCodeRunn
5.5 kg
PS C:\Users\H
```

13. How will you convert one UDT to another UDT. For example conversion of polar to cartesian system.

```
Polar p(10,5);
Cartesian c = p;
c.show();
```

```cpp
#include <iostream>
#include <cmath>
using namespace std;

class Polar {
    float r, a;
public:
    Polar(float rad, float ang) : r(rad), a(ang) {}
    float getR() { return r; }
    float getA() { return a; }
};

class Cartesian {
    float x, y;
public:
    Cartesian() : x(0), y(0) {}
    Cartesian(Polar p) {
        x = p.getR() * cos(p.getA());
        y = p.getR() * sin(p.getA());
    }
    void show() { cout << "X: " << x << ", Y: " << y << endl;
};

int main() {
    Polar p(10, 0.5);
    Cartesian c = p;
    c.show();
    return 0;
}
```

```
PS C:\Users\HP> cd "C:\Users\HP\AppD
 tempCodeRunnerFile } ; if ($?) { .\
5.5 kg
PS C:\Users\HP\AppData\Local\Temp> c
deRunnerFile.cpp -o tempCodeRunnerFi
X: 8.77583, Y: 4.79426
PS C:\Users\HP\AppData\Local\Temp>
```